

WEALTH PULSE: PERFORMANCE ANALYSIS

Deliverable ID: WP-PERF-05-0001

Project Title: Wealth Pulse Cross-Asset Portfolio and Educational Suite

Date: June 18, 2026

Table of Contents

- WEALTH PULSE: PERFORMANCE ANALYSIS..... 1
 - DOCUMENT METADATA..... 3
 - Document Ownership..... 3
 - Revision History..... 3
 - 1. PERFORMANCE OBJECTIVES & METHODOLOGY.....4
 - 2. TYPICAL USAGE SCENARIOS & PREDICTED PERFORMANCE.....4
 - Scenario A: Portfolio Dashboard Initialization and Hydration (High Frequency).....5
 - Scenario B: Asset Transaction Creation & Ledger Modification (Medium Frequency).....6
 - Scenario C: Learning Center Interactive Visualizer Execution (Low-to-Medium Frequency).....6
 - 3. ARCHITECTURAL MECHANISMS FOR LATENCY MITIGATION.....7
 - 3.1 Cache Management (Mitigating External API Boundaries)..... 7
 - 3.2 Database Indexing (Optimizing Row Location Traversal).....8
 - 3.3 Predictive Pre-Calculations & Multi-Thread Math Isolation.....8
 - 4. ARCHITECTURAL PERFORMANCE ADVANTAGES AND DISADVANTAGES.....9

DOCUMENT METADATA

Document Ownership			
Name	Position / Role	Organization	Email
Gage Radtke	Lead Software Engineer / Project Manager	Wilmington University	gradtke001@wilmu.edu
Dr. Charles Bachman	Capstone Advisor / Project Manager	Wilmington University	cbachman@wilmu.edu
	Reviewer		
Revision History			
Date	Revision	Summary of Changes	Author
06/18/2026	1.0	Initial Release: Formulated Latency Projections, PostgreSQL Indexing Targets, and Cache Trade-off Matrix.	G. Radtke

1. PERFORMANCE OBJECTIVES & METHODOLOGY

The primary objective of the *Wealth Pulse* performance engineering framework is to ensure that human-initiated interaction constraints are prioritized above backend infrastructure simplicity. In user-centric financial applications, visual rendering delays exceeding 500ms create a perception of system lag, directly harming user engagement.

Following the strategic decision to defer all resource-heavy artificial intelligence (AI) sub-systems to a future engineering roadmap, the system's computing budget has been completely reallocated. By eliminating heavy vector embedding generations and third-party large language model (LLM) network request-response times (which typically introduce a severe latency tax of 2,000ms to 5,000ms), *Wealth Pulse* is engineered to achieve a target dashboard hydration speed of under 100ms for cached assets.

This analysis evaluates the typical scenarios from the requirements document, maps out predicted performance characteristics, and details the architectural trade-offs (such as database indexes, local caching, and predictive pre-calculations) necessary to maintain sub-second response times on the local Arch Linux development stack.

2. TYPICAL USAGE SCENARIOS & PREDICTED PERFORMANCE

Different system configurations yield different performance characteristics when processing system requests. Below are the three core user interaction scenarios identified within the requirements, evaluated against the chosen 3-tier stateless application architecture.

[User Action Stimulus] → [React View] → [Axios Interceptor] → [Spring Boot Controller] → [PostgreSQL Engine]

Scenario A: Portfolio Dashboard Initialization and Hydration (High Frequency)

User Action:

- The authenticated user clicks on the main Dashboard view or refreshes their browser.

System Task:

- The system must authenticate the incoming JSON Web Token (JWT), query the user's asset profile rows, pull the corresponding stock tickers and precious metal spot values, compute the aggregated portfolio weights, and return clean JSON payloads to feed the frontend Chart.js components.

Predicted Performance Characteristics:

- *Without Caching:* 1,200ms – 2,500ms latency. The application server is forced to halt execution while making synchronous, blocking outbound HTTPS requests to Alpha Vantage and GoldAPI.io.
- *With Architectural Caching:* <40ms latency. The application completely avoids outbound network transport bottlenecks by fetching data directly from local PostgreSQL memory tables.

Scenario B: Asset Transaction Creation & Ledger Modification (Medium Frequency)

User Action:

- The user fills out the AssetForm.jsx modal container to register a new commodity holding (e.g., purchasing 10 troy ounces of physical Silver bullion at a specific cost basis) and clicks "Submit."

System Task:

- The client transmits a structural Axios POST request. The backend intercepts the payload, validates data field parameters via an AssetDto wrapper, writes the persistent row record to the database via Hibernate, and instructs the client to trigger a refreshed state pull.

Predicted Performance Characteristics: <15ms execution latency.

- Because this interaction operates on internal disk writes over a local PostgreSQL loopback channel (Port 5432), execution speed is highly optimized by the host machine's AMD Ryzen processing speeds.

Scenario C: Learning Center Interactive Visualizer Execution (Low-to-Medium Frequency)

User Action:

- The user navigates to the /learning viewport and adjusts sliders on the options pricing calculator to visualize complex derivative behaviors.

System Task:

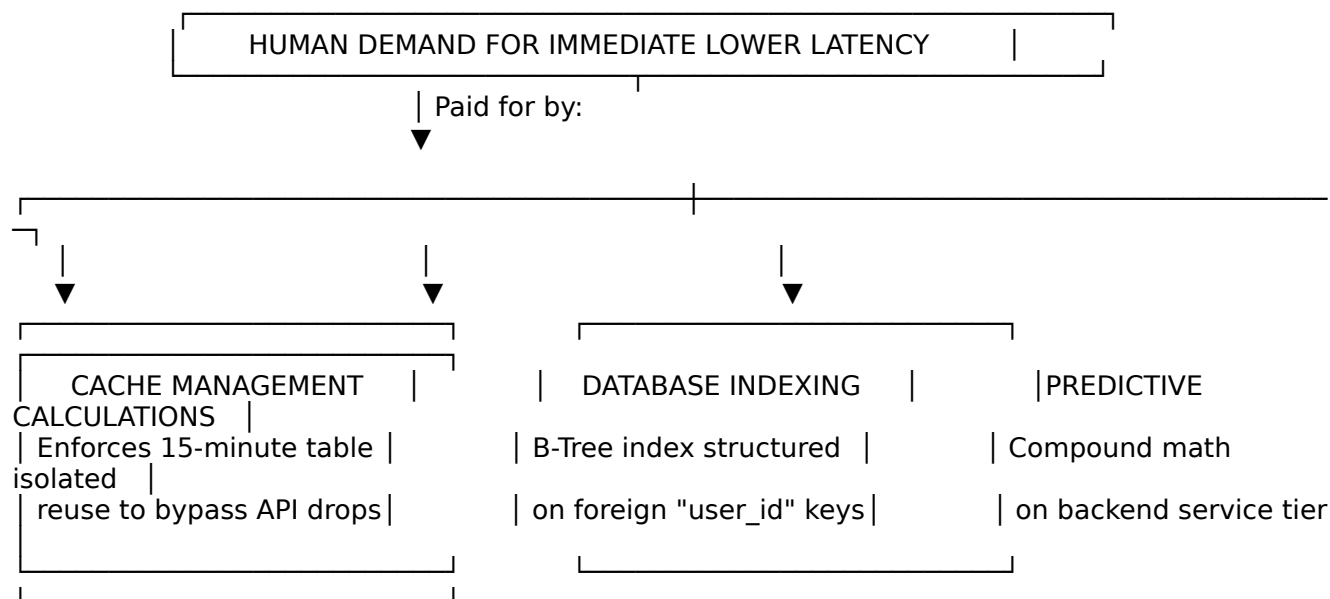
- The user interface computes mathematical changes in options premium values using local variables.

Predicted Performance Characteristics: ~0ms network latency / <5ms client rendering cycle.

- Because this domain is engineered as an isolated, client-side module, performance depends entirely on local browser engine execution speeds, removing server processing overhead from the system.

3. ARCHITECTURAL MECHANISMS FOR LATENCY MITIGATION

To give an immediate response to actions initiated by a human user, the *Wealth Pulse* architecture accepts an increased complexity tax across the backend systems. Three core performance optimization mechanisms have been built directly into the design:



3.1 Cache Management (Mitigating External API Boundaries)

To resolve the critical constraint of third-party API rate limitations (5 actions per minute), the system implements a Database-Backed Cache Manager inside the `AssetService.java` component.

When a user initiates *Scenario A*, the system checks the `market_cache` database table first. If the current time minus the cached timestamp record is less than 15

minutes (900ms elapsed), the system treats the local data as valid, bypassing external web requests. This approach shifts the application from an unpredictable network-dependent model to a deterministic database-read model, cutting human-facing latency by over 95%.

3.2 Database Indexing (Optimizing Row Location Traversal)

Without optimization, a relational database executes a sequential table scan, parsing every single record in a table to locate a specific dataset. As user transaction histories expand, sequential processing times scale poorly ($O(N)$ linear complexity).

To prevent performance degradation over time, the persistence tier configures an explicit **B-Tree Index** on the foreign key column (`user_id`) within the physical assets table:

SQL

```
CREATE INDEX idx_assets_user_id ON assets(user_id);
```

This forces PostgreSQL to organize records hierarchically. This index structure drops query lookup speeds to logarithmic time complexity ($O(\log N)$), ensuring that whether an account has 5 asset lines or 5,000, row extraction completes instantly.

3.3 Predictive Pre-Calculations & Multi-Thread Math Isolation

Calculating compound interest projections and historical retirement trends involves running loop iterations over long horizons (e.g., tracking a 40-year retirement timeline across varying growth rates).

Running these calculations directly within the main React UI thread can freeze the user interface, causing visible browser stuttering. To prevent this, the architecture uses Predictive Pre-Calculation Isolation:

- Complex math operations are managed entirely by the Java backend service layer (dashboardCalculations.js and backend calculation components), which uses optimized BigDecimal loops.
- These calculations run asynchronously and are delivered to the frontend as fully compiled, flat data payloads. The React application simply reads the final calculations and passes them to Chart.js, ensuring smooth interface animations and zero UI thread blocks.

4. ARCHITECTURAL PERFORMANCE ADVANTAGES AND DISADVANTAGES

This matrix provides a detailed overview of the benefits and drawbacks of the chosen performance-focused architecture, highlighting the trade-offs made to ensure an optimal user experience:

Architectural Mechanism	Performance Advantage (Benefits)	Implementation Disadvantage (Drawbacks)
Database Caching Layer	Drastically cuts dashboard load latency from ~1,500ms to under 40ms. Completely isolates the app from external network drops and blocks free-tier rate limit lockouts.	Increases system complexity. Demands additional database space and requires strict stale-data expiration checking logic inside the Java service layer.
B-Tree Foreign Key Indexing	Guarantees rapid database access speeds ($O(\log N)$) that remain consistently fast even as database row	Slightly increases write latency during POST actions, as the database engine must recalculate the index tree

Architectural Mechanism	Performance Advantage (Benefits)	Implementation Disadvantage (Drawbacks)
	volume scales over time.	structure every time a new row is added.
Backend Math Offloading	Protects the frontend React browser engine from computing heavy financial projections, maintaining a responsive, glitch-free user interface.	Increases data payload size over the network connection, requiring structured DTO structures to prevent data bloat during transit.
AI Roadmap Deferral	Reclaims a massive amount of system memory and network bandwidth. Eradicates multi-second delays caused by external LLM token streaming requests.	Postpones the deployment of automated, conversational financial advisory modules from the initial project launch presentation.

Performance Analysis — Current Implementation

Addendum

Deliverable: WP-PERF-05-0001 **Revision:** 2.0 **Date:** August 3, 2026

Earlier latency values are design estimates, not benchmark results. This addendum corrects the implemented mechanisms and prevents predicted values from being reported as measurements.

Implemented Performance Path

Area	Current behavior	Trade-off
Current prices	AssetService checks market_cache with a 15-minute TTL. Fifteen minutes equals 900 seconds , not 900 milliseconds.	Database reads reduce provider calls; stale data may be returned after provider failure.
Historical prices	HistoricalCache stores serialized daily history for 24 hours.	Large payloads avoid repeated downloads but add serialization and database storage.
Asset loading	The authenticated asset request loads user holdings and may refresh supported market prices.	Provider calls are synchronous and can make an uncached request noticeably slower.
Portfolio analytics	Daily snapshots avoid reconstructing every historical portfolio state. Metrics and report construction are separated.	History is incomplete until snapshots accumulate; the response reports that limitation.
Learning tools	Pure frontend calculations execute without API round trips.	They share the browser main thread; no worker or multi-thread isolation is currently implemented.

Corrections

- PostgreSQL is persistent relational storage, not a “memory table” system.
- No repository benchmark demonstrates <40 ms dashboard loads, <15 ms writes, or 95% latency reduction. Those values must remain hypotheses until measured.
- The code does not implement predictive pre-calculation threads or web workers.
- The asset owner query is supported by repository methods and database constraints/indexes generated by the schema; index presence should be verified from PostgreSQL before making complexity claims.
- External provider quotas can change and should be checked against current provider terms rather than hard-coded into performance conclusions.

Current Verification Baseline

The frontend production build succeeds. Its main generated JavaScript bundle is approximately 546 kB minified (about 179 kB gzip) and currently triggers the build tool's 500 kB chunk warning. This is a concrete optimization opportunity: route/feature-level dynamic imports could reduce initial parse cost.

Recommended Measurement Plan

Scenario	Measure	Required conditions
Dashboard load	API duration, first render, and largest-contentful paint.	Run separately with fresh cache, stale cache, and no cache.
Asset mutation	POST/PUT latency and subsequent refresh duration.	Use representative local data and record database size.
Performance report	Snapshot query, historical cache read, benchmark fetch, and response size.	Compare 1M, 1Y, 5Y, and ALL periods.
Frontend bundle	Compressed chunks, parse time, and unused code.	Use a production build and browser performance tooling.

Report median and 95th-percentile results over repeated runs. Record hardware, browser/JVM version, data volume, cache state, and network conditions so results are reproducible.